

Mastering Spark Notebooks and Capacity Optimization

in Microsoft Fabric

Thank you sponsors!



redgate

twintos wortell *infoSupport*
Solid Innovator



INSPARK

macaw

Your guide in the Era of AI.

Rubicon

Consulting & Technology

ilionx



Data Saturdays

BRANDER



company

Just Blindbæk

- **Principal Architect at twoday**
 - Pre-sales, workshops, events, marketing
 - Internal practice development
 - Academy: External training
- **Microsoft Data Platform MVP**
- **Found and organizer of**
 - Danish Microsoft BI Community (MsBIP.dk)
 - Power BI UG Denmark (PowerBI.dk)
 - Data Platform Next Step, Power BI Next Step and Power BI Cruise

 [linkedin.com/in/blindbaek/](https://www.linkedin.com/in/blindbaek/)

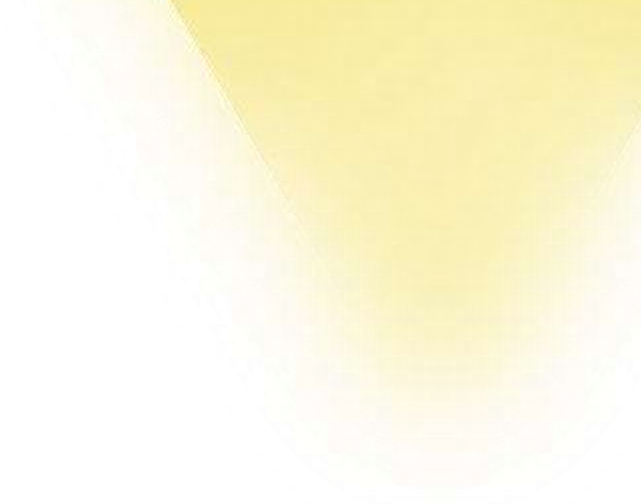


Slides



Mastering Spark Notebooks and Capacity Optimization

1. How Spark Compute Works
2. Capacity Consumption in Practice
3. Optimizing Spark Notebooks
4. Tips to Reduce CU Consumption
5. Wrap-up & Q&A

A yellow circular graphic with a gradient, partially visible in the top right corner of the slide.

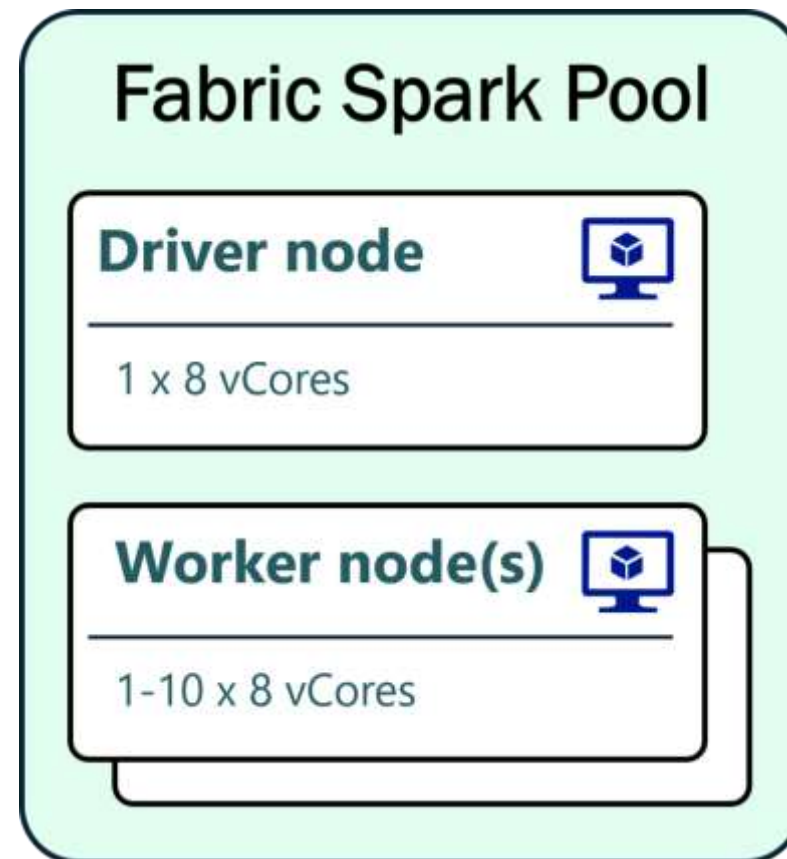
How Spark Compute Works

How Spark Compute Works

- Spark operates as a Platform-as-a-Service (PaaS) engine within the broader Software-as-a-Service (SaaS) ecosystem
- While it is serverless, you still “pay” for the compute resources as long as they remain allocated - even if you’re running lightweight operations that only use a fraction of the allocated power.
- Creating a Spark pool is free – you are only billed when your Spark job/notebook is active
- Your “server” in Fabric is a cluster of multiple machines (nodes), and in Fabric, this is called a Pool
 - Driver Node: The central coordinator that receives and manages user requests
 - Executor Nodes: The actual workers executing the tasks

Understanding the Default Starter Pool

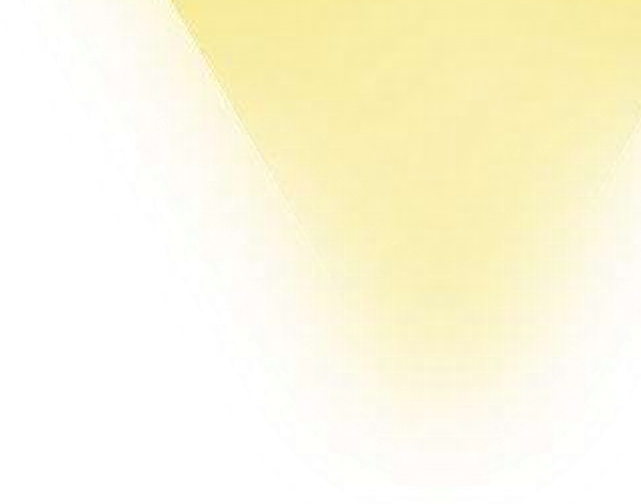
- By default, Fabric assigns a Medium-sized Starter Pool, designed for fast startup times (within a few seconds)
 - Autoscaling: 1 to 10 nodes
 - Minimum allocation: 1 driver + 1 worker
 - Each node: 8 vCores
 - Minimum total allocation: 16 vCores (8 vCores x 2 nodes)



vCores mapping to Capacity Unites

- 1 CU = 2 Spark vCores
- Starter Pool minimum total allocation: 16 vCores
- Then we need at least a F8 ?

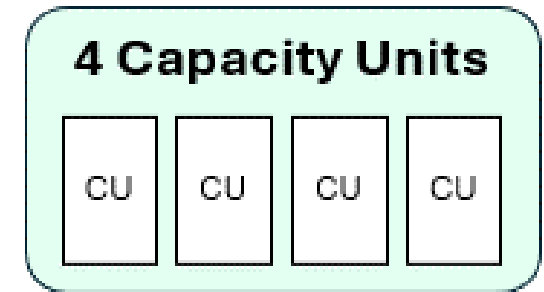
Capacity SKU	Base CUs	Base Spark vCores
F2	2 CUs	4 vCores
F4	4 CUs	8 vCores
F8	8 CUs	16 vCores
Trial Capacity	64 CUs	128 vCores

A yellow circular graphic is located in the top right corner of the slide, partially cut off by the edge.

Capacity Consumption in Practice

Spark Bursting Explained

- Allows temporary overuse of capacity
- Helps with concurrent workloads and the maximum cores available for a single Spark job
- Default is a 3x burst multiplier
- Bursting Factor for F4 Capacity
 - 3x burst multiplier
 - 12 Capacity Units
 - 24 Spark vCores

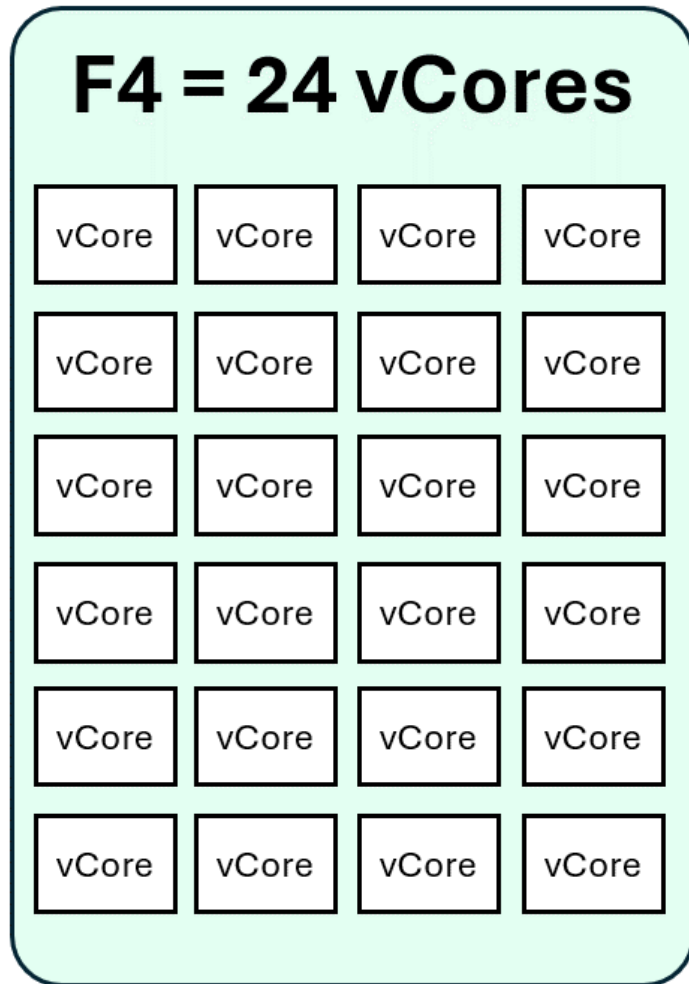


Spark Bursting Factors and Max vCores

Capacity SKU	Base CUs	Base Spark vCores	Burst Multiplier	Max vCores with Bursting
F2	2 CUs	4 vCores	5x	20 vCores
F4	4 CUs	8 vCores	3x	24 vCores
F8	8 CUs	16 vCores	3x	48 vCores
Trial Capacity	64 CUs	128 vCores	1x	128 vCores

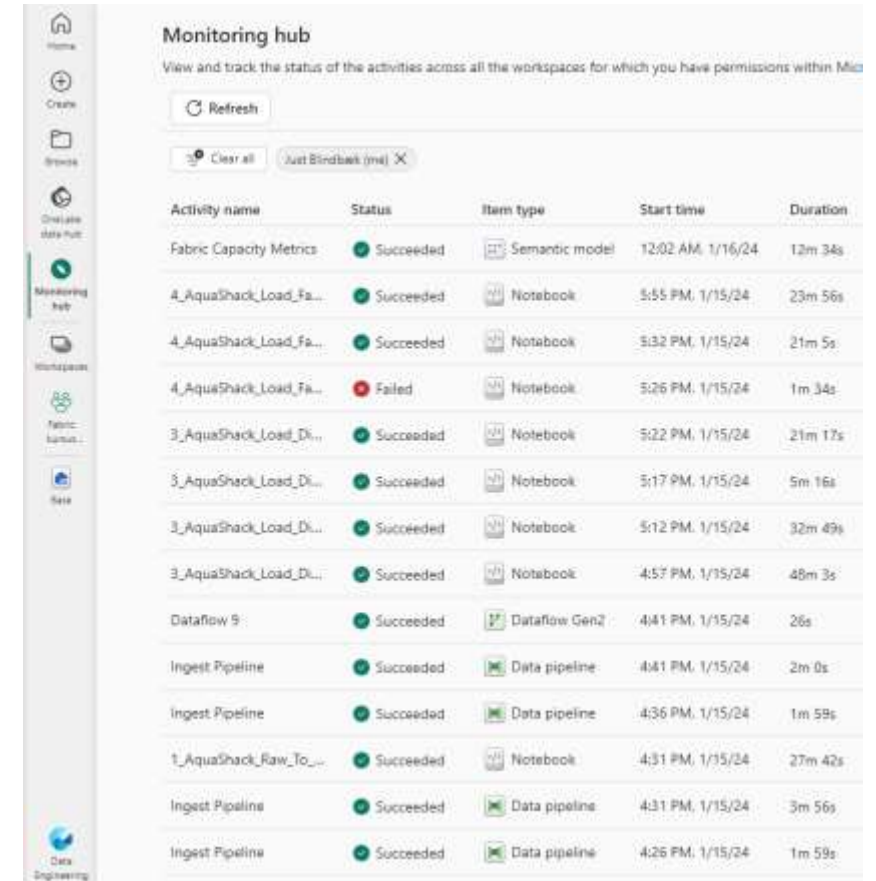
- Exceptions
 - Trial capacities do not offer bursting
 - F2 has a 5x burst factor, allowing it to scale from 4 vCores to 20 vCores
- It's possible to disable job-level bursting

How Notebooks Use Capacity



Monitor CU Consumption in the Monitor Hub

- Fabric provides built-in monitoring tools to help optimize Spark pool usage.
- Add Allocated Resources as a column in Monitor Hub to see how many executors and cores were used
- Go a step deeper and click on the activity name and then display the Spark configuration details, including
 - Number of executors over time
 - vCores used for drivers and workers
 - Execution breakdowns (duration, resource allocation, etc.)

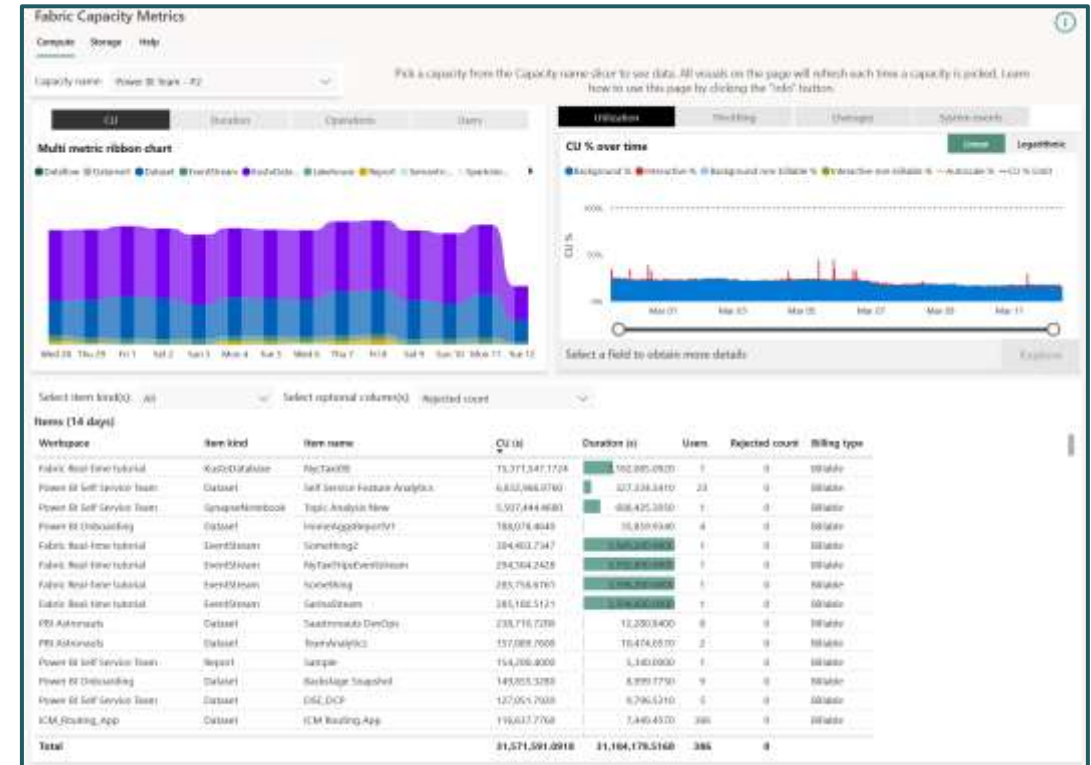


The screenshot shows the 'Monitoring hub' interface in Azure. It features a sidebar with navigation icons for Home, Create, Browse, OneLake data hub, Monitoring hub (selected), Workspaces, Fabric, and Data. The main content area displays a table of activities with columns for Activity name, Status, Item type, Start time, and Duration. The table lists various activities such as 'Fabric Capacity Metrics', 'AquaShack_Load_Fa...', and 'Dataflow 9', with their respective statuses (Succeeded or Failed) and durations.

Activity name	Status	Item type	Start time	Duration
Fabric Capacity Metrics	Succeeded	Semantic model	12:02 AM, 1/16/24	12m 34s
4_AquaShack_Load_Fa...	Succeeded	Notebook	5:55 PM, 1/15/24	23m 56s
4_AquaShack_Load_Fa...	Succeeded	Notebook	5:32 PM, 1/15/24	21m 5s
4_AquaShack_Load_Fa...	Failed	Notebook	5:26 PM, 1/15/24	1m 34s
3_AquaShack_Load_Di...	Succeeded	Notebook	5:22 PM, 1/15/24	21m 17s
3_AquaShack_Load_Di...	Succeeded	Notebook	5:17 PM, 1/15/24	5m 16s
3_AquaShack_Load_Di...	Succeeded	Notebook	5:12 PM, 1/15/24	32m 49s
3_AquaShack_Load_Di...	Succeeded	Notebook	4:57 PM, 1/15/24	48m 3s
Dataflow 9	Succeeded	Dataflow Gen2	4:41 PM, 1/15/24	26s
Ingest Pipeline	Succeeded	Data pipeline	4:41 PM, 1/15/24	2m 0s
Ingest Pipeline	Succeeded	Data pipeline	4:36 PM, 1/15/24	1m 59s
1_AquaShack_Raw_To...	Succeeded	Notebook	4:31 PM, 1/15/24	27m 42s
Ingest Pipeline	Succeeded	Data pipeline	4:31 PM, 1/15/24	3m 56s
Ingest Pipeline	Succeeded	Data pipeline	4:26 PM, 1/15/24	1m 59s

Monitor CU Consumption in Capacity Metrics App

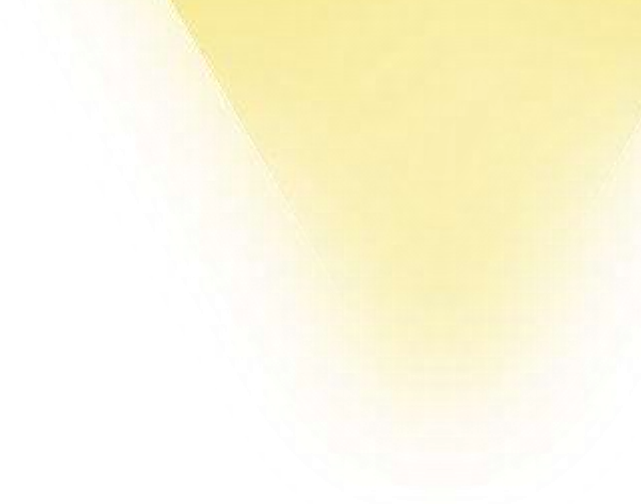
- Gain tenant-wide visibility into capacity usage across all Fabric workloads.
- Identify resource usage trends to optimize autoscale and mitigate throttling impacts.
- Compare preview and production workloads for data-driven capacity sizing decisions.
- Zoom in on workload operations and artifacts with detailed granularity down to 30 seconds.
- Monitor real-time operations and the impact of long-running jobs on capacity limits.
- Analyze user experience to plan efficient scale-ups and optimize resource allocation.



<https://learn.microsoft.com/en-us/fabric/enterprise/metrics-app>

Interactive vs. Scheduled Notebooks

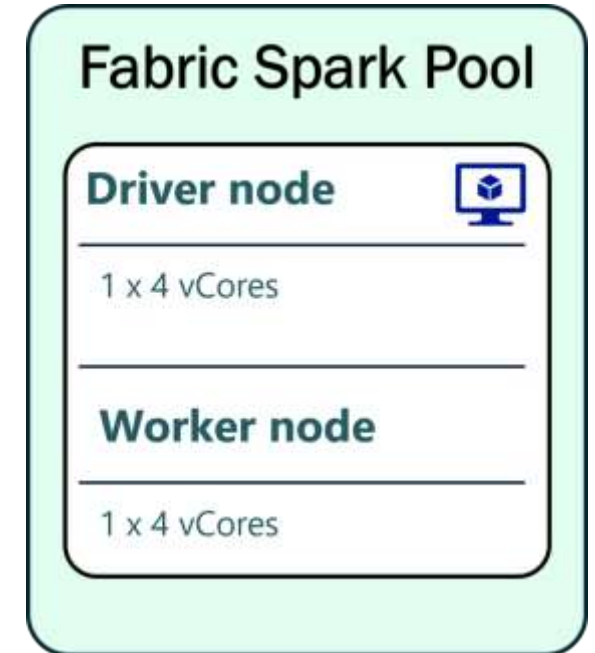
- Interactive:
 - Default idle timeout is 20 minutes. Change in workspace settings.
 - Configuring auto shut-down to prevent idle CU burn
 - Interactive notebooks do not queue - they return an error instead
- Scheduled
 - Queue limit is equal to the Capacity SKU size
 - Jobs in the queue will retry automatically when resources become available
 - Automatically start and stop when the job completes, consuming CU only for the duration of execution.

A yellow circular graphic with a gradient, partially visible in the top right corner of the slide.

Optimizing Pool Configurations

Single-node Spark Pool

- By setting autoscale to 1 for both minimum and maximum nodes, the driver and executor run on the same node, reducing the vCore requirement.
- Each session now uses only 8 vCores
- An F4 capacity can now support up to 3 active sessions instead of just 1



[illegible]

vCore

vCore

vCore

vCore

vCore

vCore

vCore

vCore

vCore

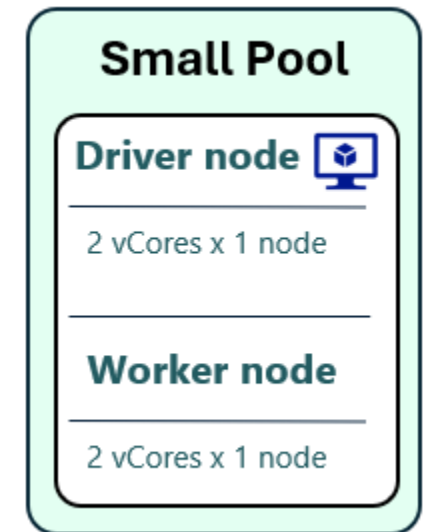
vCore

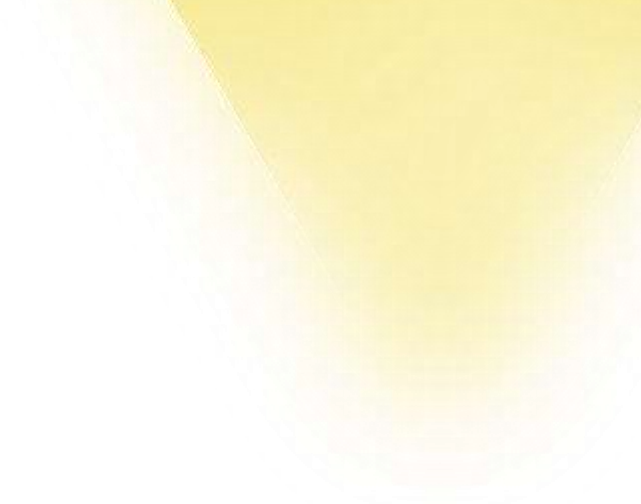
vCore

vCore

Custom Small Pool

- A Small Pool, forced to 1 node, uses only 4 vCores
- An F4 capacity can now run up to 6 active sessions
- Limited compute power per session
- Longer startup time compared to Starter Pools



A yellow circular graphic is located in the top right corner of the slide, partially cut off by the edge.

Optimizing Session Sharing

High Concurrency Mode

- Multiple notebooks can share the same Spark session instead of launching separate compute clusters
- High Concurrency Mode is user-specific - sessions cannot be shared across different users.
- All notebooks must use the same default Lakehouse configuration.
- Enable High Concurrency Mode in Workspace Settings

[illegible]

vCore

vCore

vCore

vCore

vCore

vCore

vCore

vCore

vCore

vCore

vCore

vCore

When to use High Concurrency Mode

- Running multiple notebooks interactively – Reduces session startup time.
- Iterative development where you frequently test different scripts.

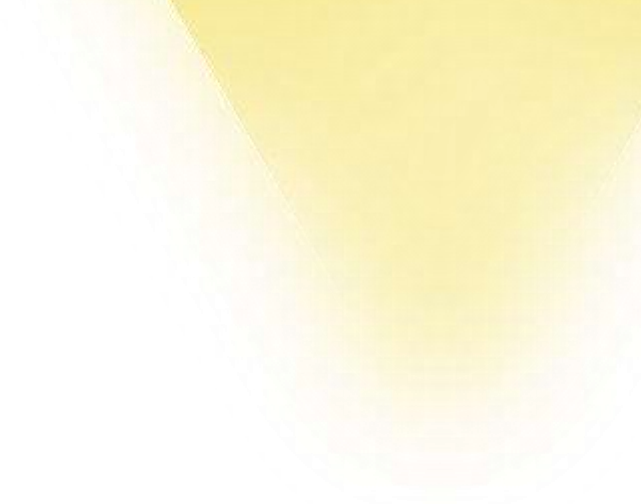
Use Case	Standard Session	High Concurrency Mode
Running a single notebook	✓ Best choice	✗ Not necessary
Multiple notebooks running concurrently	✗ Each creates its own Spark cluster (high CU usage)	✓ Notebooks share the same compute session
Interactivity & performance	⚠ Slower startup time for each new session	✓ Fast execution if a session is already active
CU optimization on limited capacity	✗ Higher CU burn	✓ More efficient CU usage

Using High Concurrency Mode in Pipelines

- Reduces redundant session startups, optimizing CU usage.
- Speeds up execution by attaching notebook activities to an existing session.
- Steps to configure session sharing in Pipelines
 - Add a Notebook activity to the Pipeline and select your notebook.
 - In Advanced settings, locate the Session tag field.
 - Assign a custom session tag (e.g., "pipeline_session").
 - Use the same session tag for all subsequent notebook activities that should share the session.

RunMultiple()

- Enables a parent notebook to trigger child notebooks within the same session
- Orchestrating multiple notebooks
- Execution follows a defined DAG (Directed Acyclic Graph) structure
- Defines dependencies between notebooks
- Enables parallel execution where possible

A yellow circular graphic with a gradient, partially visible in the top right corner of the slide.

Tips to Reduce CU Consumption

Tips to Reduce Spark CU Consumption

- Single-node Spark Pools
- Lower auto-shutdown
- High Concurrency Mode for interactive work
- Monitor sessions
- RunMultiple for orchestration

Use Python Notebooks for Small Workloads

- Spark is a distributed system - overhead can be costly for simple tasks
- Try Python notebooks
 - Default configuration: 2 vCores
 - Lower CU consumption
 - Supports Polars and delta-rs for efficient data processing.

Autoscale Billing for Spark

- A flexible, pay-as-you-go billing model for Spark workloads in Microsoft Fabric.
- With this model enabled, Spark jobs use dedicated serverless resources instead of consuming compute from Fabric capacity.
- This serverless option optimizes cost and provides scalability without resource contention.
- The billing for Spark jobs is based solely on the compute used during job execution

Wrap-up: 3 takeaways

- Understand how Spark sessions consume capacity
- Optimize with Pool Configurations
- Optimize with Session Sharing & Orchestration
- Blogpost series:
 - <https://justb.dk/blog/2025/02/fabric-spark-notebooks-and-cu-consumption/>



Thank you sponsors!



redgate

twintos wortell *infoSupport*
Solid Innovator



INSPARK

macaw

Your guide in the Era of AI.

Rubicon

Consulting & Technology

ilionx



Data Saturdays

BRANDER



company

Rate Saturday Holland



Review to win
With custom Data Saturdays lego stickers!

Rate This session

